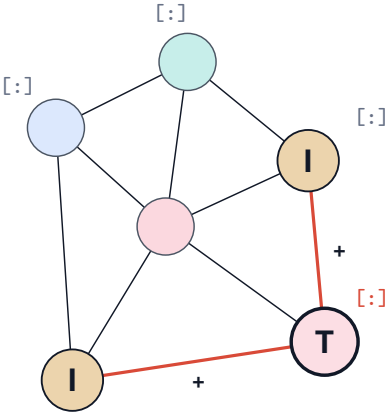
A Survey of Adversarial Learning on Graphs

Attacks, defenses, metrics, and open problems

Krzysztof Czerwionka Przemysław Fuchs

TGS
Wydział Matematyki, Informatyki i Mechaniki
Uniwersytet Warszawski

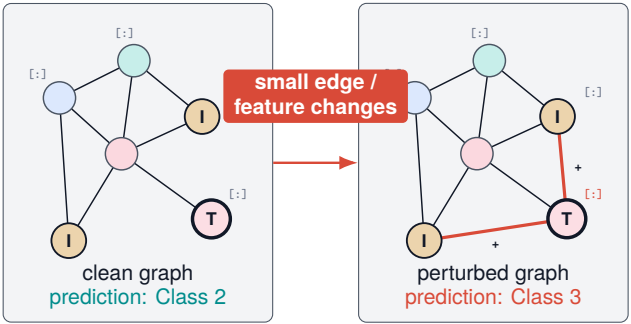
May 6, 2026



small graph perturbations under constraints

Chen et al., *A Survey of Adversarial Learning on Graphs*, arXiv:2003.05730v3

Why adversarial learning on graphs matters



Deep graph learning is powerful and vulnerable

Small changes to edges or features can flip a prediction.

Typical tasks

- node classification
- link prediction
- graph classification

What this survey is trying to do

Unified formulations
Put many paper-specific definitions into shared optimisation language.

Attack taxonomies
Separate knowledge, goal, capability, strategy, manipulation, algorithm, and target task.

Defense taxonomies
Organise defenses by where they intervene: data, model, objective, training, or monitoring.

Metrics and open problems
Treat evaluation and research gaps as first-class survey content.

Intro → Setup → Attacks → Defenses → Metrics → Discussion → Conclusion **survey order**

Graph setup used in the paper

Minimum notation	
$\mathbf{G} = (\mathbf{V}, \mathbf{E})$	graph with nodes and edges
$\mathbf{A}$	adjacency matrix, $A \in \mathbb{R}^{N \times N}$
$\mathbf{X}$	node-feature matrix, $X \in \mathbb{R}^{N \times F}$

Default scope in this survey

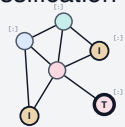
Unless stated otherwise, the paper works with simple graphs: undirected, unweighted, static, and homogeneous.

Target instance t can be a node, an edge, or a whole graph depending on the downstream task.

Tasks and learning settings

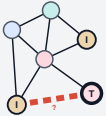
Node-level

node classification



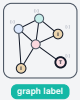
Link-level

link prediction



Graph-level

graph classification



Generic supervised objective

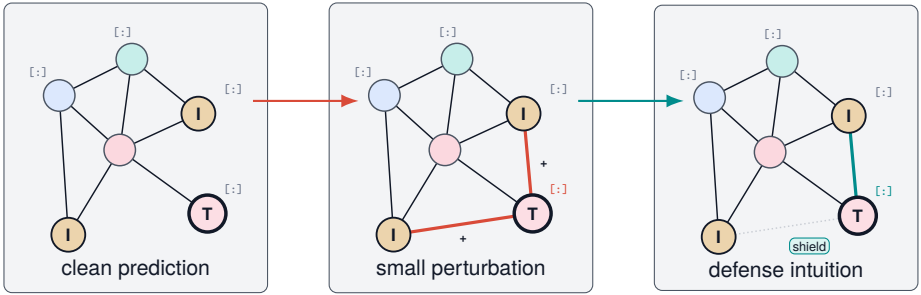
$$L = \frac{1}{|V_L|} \sum_{v_i \in V_L} \mathcal{L}\big(f_\theta(G, X), y_i\big)$$

V_L: labelled nodes

$f_\theta(G, X)$: model output

y_i: target label

The toy example to keep in mind



topology change | feature change | budget Δ | imperceptibility Q

A unified formulation of attack

Outer maximisation, inner training

Attack chooses a perturbation

$$\max_{\hat{G} \in \Psi(G)} \sum_{t_i \in T} \mathcal{L}\big(f_{\theta^*}(\hat{G}^{t_i}, X, t_i), y_i\big)$$

Model trains on the poisoned graph

$$\theta^* = \arg\min_{\theta} \sum_{v_j \in S_L} \mathcal{L}\big(f_\theta(\tilde{G}^{v_j}, X, v_j), y_j\big)$$

Read the equation as

- choose a perturbed graph $\hat{G}$
- maximize target loss on T
- account for retraining on $\tilde{G}$

Constraint

$$Q(\hat{G}, G) < \epsilon$$

stay close to the clean graph

Attack taxonomy in one view

Knowledge
white / grey / black

Goal
availability / integrity

Capability
poisoning / evasion

Strategy
topology / feature

Manipulation
add / remove / rewire

Algorithm
gradient / search / RL

Task
node / link / graph

An attack is a combination of choices across axes.

Why this taxonomy matters

A single method can be white-box, targeted, poisoning, topology-based, rewiring-based, gradient-driven, and node-classification focused.

What attackers know and what they want

Model knowledge

- white-box: architecture, parameters, gradients
- grey-box: partial model access or surrogate
- black-box: limited queries or little model knowledge

Data knowledge perfect → moderate → minimal

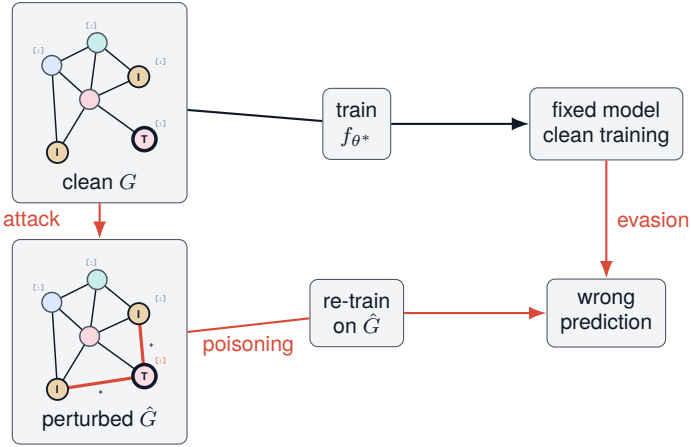
Goal distinctions

scope	availability vs integrity
target set	targeted vs general
error	specific label vs any wrong label

Terminology caveat

In this survey, targeted means target range, not necessarily a specific target label.

Poisoning versus evasion



Training line changes

poisoning: train on $\hat{G}$

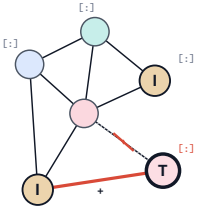
evasion: train on G

What can be perturbed and how

What is changed?
topology · feature · hybrid

What operation is allowed?
add · remove · rewire

How is it found?
gradient · genetic · RL · generative



red edge = added edge
red dotted edge = removed edge
red [:] = changed feature

Budget Δ

Topology $$\sum_{u<v} |A_{uv} - A'_{uv}| \leq \Delta$$

Feature $$\sum_{u,j} |X_{uj} - X'_{uj}| \leq \Delta$$

Δ limits how many graph entries the attacker may change.

Representative attack methods

Representative anchors from the survey table

Method	Why it is on the slide
Nettack	early node-classification attack; topology and feature perturbations
Meta-Self / PGD	optimisation-based poisoning and topology attacks
RL-S2V	reinforcement-learning attack for node and graph classification
IGA	link-prediction attack using graph auto-encoder gradients
ReWatt	graph-level rewiring attack; useful for stealth intuition

Representative examples only – see the survey tables for the fuller comparison.

What the attack literature still misses

Unnoticeability
Low budget is not the same as low detectability.

Knowledge assumptions
Perfect graph and label knowledge is often unrealistic.

Scalability
Many methods remain tied to small benchmark graphs.

Physical realism
Real systems are not simple, static, ideal graph inputs.

Transition: switch perspective from choosing perturbations to resisting them.

A unified formulation of defense

Defender minimises robust loss

$$\min_{\substack{\tilde{G}\in\Psi(G)\cup G \\ \text{clean or perturbed}}} \sum_{t_i\in T} \tilde{\mathcal{L}}\big(\underbrace{\tilde{f}_{\theta^*}}_{\text{defended model}}(\tilde{G}^{t_i}, X, t_i), y_i\big)$$

Training objective

$$\theta^* = \arg\min_{\theta} \sum_{v_j\in S_L} \tilde{\mathcal{L}}\big(\tilde{f}_\theta(\tilde{G}^{v_j}, X, v_j), y_j\big)$$

Goal: low loss on clean or perturbed graphs.

Attack

Choose perturbations to maximise loss.

Defense

Train a model to minimise loss under perturbations.

Same graph setting. Opposite optimisation direction.

Defense taxonomy in one view

Preprocessing clean data first	**Structure** robust aggregation	**Adversarial** train against attacks
Objective loss / regularisation	**Detection** monitor / certify	**Hybrid** combine families

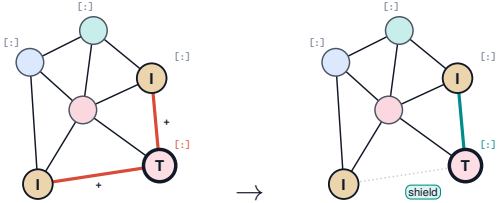
Intervention question

Where does the defense act: before learning, inside the model, during training, in the objective, or after deployment?

Clean the graph or change the model

Preprocessing-based defense

Remove suspicious edges before training.



drop edges

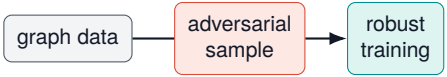
Structure-based defense

Redesign aggregation or convolution so perturbed messages matter less.

Train against attacks or change the objective

Adversarial-based defense

Train with adversarial goals or adversarial examples.



VAT/BVAT GraphDefense

Objective-based defense

Redesign the loss or regularisation for robustness.

Objective sketch

$$\mathcal{L}_{robust} = \mathcal{L}_{task} + \lambda \mathcal{R}_{smooth}$$

r-GCN/VPN SCEL/SD

The survey notes that adversarial- and objective-based categories partially overlap.

Detect, certify, and combine

Detection / certification

Monitor attacks or certify robustness under a perturbation space.

Hybrid

Combine preprocessing, structure, adversarial training, objective changes, or detection.

Representative methods

Family	Examples and main idea
Preprocess	Drop suspicious low-similarity links
Structure	RGCN / AGCN: robust aggregation or edge dithering
Adversarial	VAT / BVAT: train against perturbations
Certification	Bound worst-case node robustness
Hybrid	DefNet / Global-AT: combine model and training changes

Most defense work still centres on node classification.

Why metrics deserve their own section

Effectiveness
Does it work?
Acc · F1 · ASR · CM · ADR

Efficiency
What did it cost?
budget Δ · AML · training cost

Imperceptibility
Is it noticeable?
similarity · stats · constraints

Task $\rightarrow$ metric map

classification	Acc · F1 · FNR · FPR
ranking / link prediction	AUC · AP · MRR · Hits@K
clustering / communities	NMI · ARI · modularity

The few metrics worth remembering

Attack Success Rate

$$ASR = \frac{\text{successful attacks}}{\text{attacks}}$$

Average Modified Links

$$AML = \frac{\text{modified links}}{\text{attacks}}$$

Average Defense Rate

$$ADR = \frac{ASR_{\text{with defense}}}{ASR_{\text{without defense}}} - 1$$

Optional detail

CM: margin to the correct class. Λ: statistical similarity between clean and attacked graphs.

Open problems for attacks

Unnoticeable attacks
Effective perturbations should also preserve important graph properties.

Weaker knowledge
Practical attacks should need less graph, label, and model access.

Scalable attacks
Large graphs make full-gradient or full-candidate methods expensive.

Real-world graphs
Move beyond simple, static, undistorted graph inputs.

Open problems for defenses and evaluation

Defense

- defense beyond node classification
- efficient defense at scale
- certification of robustness

Evaluation

- better cost metrics
- better imperceptibility metrics
- better metric selection by scenario

Counting modified edges is often too crude: adding and removing an edge may have different real costs.

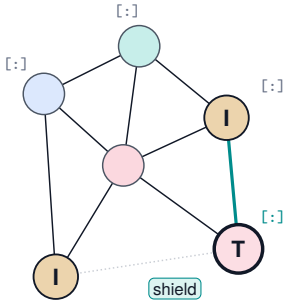
What the paper leaves us with

This survey's main contribution is structure

1. Attacks are constrained perturbation problems on graphs.
2. Defenses intervene at the data, model, objective, training, or monitoring level.
3. Metrics and assumptions matter as much as raw performance.

unified attack $\leftrightarrow$ unified defense

A Survey of Adversarial Learning on Graphs

Reliable graph learning needs better models and better threat models.

Krzysztof Czerwionka and Przemysław Fuchs